

WAREHOUSE INVENTORY MANAGEMENT AND OPTIMIZATION SYSTEM

A Project Report Submitted in the partial fulfillment of the requirements for the
award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

MOKSHITHA : 2520030396

R. ABHISHIKTHA : 2520030185

FIRSTNAME LASTNAME : 25YYYYYY

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering

2026–27

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

The Project Report entitled “Warehouse Inventory Management and Optimization System” is a record of bonafide work of the project team submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The work presented in this report describes the design and implementation of a warehouse inventory system that combines six algorithmic modules, CSV-based inventory storage shared by a website and a terminal application, a Java backend, and a browser-based Warehouse Control Center.

MOKSHITHA – 2520030396

R. ABHISHIKTHA – 2520030185

FIRSTNAME LASTNAME – 25YYYYYY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “Warehouse Inventory Management and Optimization System” is a bonafide work done and submitted by the project team in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Computer Science and Engineering during the academic year 2026–27.

Signature of the Guide

Signature of the HOD

(Dr. V. Sireesha)

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her guidance, encouragement, and valuable suggestions throughout the development of this project.

We also thank Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic support and facilities required to complete this work. We are grateful to the department and all faculty members who contributed directly or indirectly to our learning and project development.

Finally, we thank our team members, peers, and everyone who supported the testing and refinement of the Warehouse Control Center.

ABSTRACT

Warehouse operations depend on accurate inventory information, efficient search, timely replenishment, balanced stock distribution, and reliable identification of operational risks. When inventory grows to a large scale, manual processing becomes slow and leads to delayed decisions and inefficient stock movement.

This project presents a Warehouse Inventory Management and Optimization System implemented as a single Java program with a browser interface served on localhost. The live inventory is a CSV file of 2,804 product records across five warehouses; a generated dataset of 10,000,000 records (one crore) is used to demonstrate large-scale processing. The website and the terminal application read and write the same CSV file, so a change made in one appears in the other. Users log in with role-based accounts and can add, update, delete, and search products.

The six course modules are mapped to six warehouse features. Inventory Search uses Aho-Corasick, KMP and edit-distance typo correction (Modules 1 and 2); Replenishment uses 0/1 knapsack dynamic programming to choose the best restocks within a budget (Module 3); Warehouse Balancing uses Edmonds-Karp maximum flow to move surplus stock to shortages (Module 4); Risk and Priority ranks products by stock-out risk and uses greedy set cover, an approximation algorithm, to decide which suppliers to contact first (Module 5); Cycle Counting uses reservoir sampling, and Large Inventory Processing uses parallel aggregation, randomised quickselect and streaming string search (Module 6).

All algorithms are implemented by hand without `java.util` collections inside the algorithm engine. The system passed 23 automated algorithm and integration tests, and processed the 10,000,000-record dataset in a few seconds per operation on a single-core test machine.

TABLE OF CONTENTS

1	Introduction
1.1	Overview
1.2	Problem Statement
1.3	Scope of the Project
1.4	Objectives
2	System Requirements
2.1	Functional Requirements
2.2	Non-Functional Requirements
2.3	Software and Hardware Requirements
2.4	Dataset Requirements
3	System Design
3.1	High-Level Architecture
3.2	Data Flow
3.3	Module Mapping
3.4	Algorithm-to-Feature Mapping
4	Implementation
4.1	Module-Wise Implementation
4.2	Front-End Logic and UI Rendering
4.3	API Integration
4.4	CSV Data Processing
4.5	Pseudocode
5	Results

5.1 Test Environment

5.2 Functional Results

5.3 Performance Observations

5.4 Screenshots and Analysis

6 Challenges and Limitations

6.1 Issues Faced During Development

6.2 Solutions Applied

6.3 Limitations of the System

7 Conclusion and Future Work

7.1 Conclusion

7.2 Future Work

7.3 Skills Learned During Development

References

LIST OF FIGURES

- 1.1 High-Level Architecture of the Warehouse Inventory System
- 1.2 Inventory Data Flow Between CSV, Java Backend, and Web Dashboard
- 3.1 Warehouse Control Center Architecture
- 3.2 Algorithm-to-Module Mapping
- 4.1 Inventory Search Processing Flow
- 4.2 Replenishment Processing Flow
- 4.3 Warehouse Balancing Process
- 4.4 Risk and Priority Processing Flow
- 4.5 Cycle Counting and Large Inventory Processing Flow
- 5.1 Dashboard Screen
- 5.2 Inventory Screen with Typo-Tolerant Search
- 5.3 Replenishment Screen
- 5.4 Warehouse Balancing Screen
- 5.5 Risk and Priority Screen
- 5.6 Cycle Counting Screen
- 5.7 Large Inventory Processing Screen
- 5.8 String Algorithm Lab Screen

1. INTRODUCTION

1.1 Overview

The Warehouse Inventory Management and Optimization System provides a practical control center for large-scale inventory operations. A single Java program contains the algorithm engine, the CSV storage layer, an HTTP server, and a terminal application, while a browser page (HTML, CSS and JavaScript) provides the Warehouse Control Center. The live inventory holds 2,804 product records spread over five warehouses (Hyderabad, Mumbai, Delhi, Chennai and Kolkata), and a separate generated dataset of 10,000,000 records is used for large-scale processing.

The interface contains a Dashboard, Inventory (with search and add/update/delete), Replenishment, Balancing, Risk & Priority, Cycle Count, Large Data, String Lab, and a Modules page. Each page calls a real backend operation; no results are hard-coded. The purpose is to turn algorithmic processing into understandable warehouse operations rather than isolated academic demonstrations.

1.2 Problem Statement

Large inventory datasets create challenges in locating products quickly (including when names are misspelled), deciding what stock should be replenished within a limited budget, moving surplus stock to warehouses that are short, identifying which products and suppliers need attention first, choosing a small random set of items for physical cycle counting, and processing millions of records efficiently. A useful system must combine scalable data processing with practical decision-support features.

1.3 Scope of the Project

The scope includes CSV-based inventory storage shared by the website and the terminal, a Java implementation with a hand-built algorithm engine, user login with admin and staff roles, product add/update/delete, live synchronisation between storage and the dashboard, a 10,000,000-record dataset generator, and six decision features: Inventory Search, Replenishment, Warehouse Balancing, Risk

& Priority, Cycle Counting, and Large Inventory Processing. Automated tests and result files are produced for each module.

1.4 Objectives

- Provide fast, typo-tolerant inventory search by SKU, name, category, warehouse, and supplier.
- Apply the six course modules (text processing, string algorithms, dynamic programming, network flow, approximation, randomised and parallel algorithms) to real warehouse tasks.
- Choose the best set of products to restock within a given budget.
- Decide how stock should move between warehouses to fix shortages and surpluses.
- Prioritise products, warehouses and suppliers using a defined risk score and an approximation algorithm.
- Generate unbiased random cycle-count samples and record count variances.
- Process a dataset of 10,000,000 records efficiently using streaming and parallel methods.
- Keep the terminal application, Java backend, CSV storage, and browser dashboard on one shared data file.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

- Load, validate and save inventory records in CSV storage (atomic writes), with login for admin and staff users.
- Add, update and delete products from the website and the terminal, rejecting invalid values and duplicate SKU-and-warehouse pairs.
- Search inventory by SKU, name, category, warehouse and supplier, tolerating spelling mistakes and suggesting corrections.

- Produce a budget-constrained replenishment plan and, on request, apply it to the stock.
- Compute warehouse-to-warehouse stock transfers within route capacities, calculate risk priorities, and choose suppliers to contact first.
- Generate cycle-count samples and record counted quantities and variances.
- Generate and process a 10,000,000-record dataset using streaming, parallel and randomised algorithms.
- Expose all operations to the dashboard through API endpoints and refresh the dashboard when the CSV file changes.

2.2 Non-Functional Requirements

- Correctness: every algorithm is checked against brute-force or textbook results by automated tests
- Responsiveness: search and dashboard calls return within a fraction of a second on the live inventory
- Scalability: large-data operations handle 10,000,000 records by streaming the file rather than loading it all
- Maintainability: algorithm engine, storage, web server and terminal are separate parts of one source file; the web page is a separate HTML file
- Clear presentation of results: every module shows a table, summary figures and an explanation note
- Reliable CSV handling: invalid lines are skipped, user text is sanitised, and writes are atomic

2.3 Software and Hardware Requirements

The project requires a Java Development Kit (version 11 or newer), a modern web browser, and VS Code with its integrated terminal. No external libraries, build tools, Node.js, or database server are needed. The program is compiled with javac and run locally; the website is served at <http://localhost:8080>. The 10,000,000-record dataset occupies about 0.95 GB of disk space.

2.4 Dataset Requirements

The live inventory file `inventory.csv` has ten columns: `id`, `sku`, `name`, `category`, `warehouse`, `supplier`, `qty` (quantity on hand), `min` (minimum stock level), `price` (unit price) and `demand` (units sold per day). It contains 2,804 records for about 700 distinct SKUs, each stocked in three to five of the five warehouses, across six categories and twelve suppliers. A SKU may exist in several warehouses but only once per warehouse. A second file, `inventory_large.csv`, uses the same columns and is generated by the program with 10,000,000 records for large-scale testing. The data is synthetic sample data created for this project.

3. SYSTEM DESIGN

3.1 High-Level Architecture

The architecture is: Browser (Warehouse Control Center) → Java HTTP server (`com.sun.net.httpserver`, REST-style API) → algorithm engine and business logic → `inventory.csv`. The terminal application calls the same Java functions directly, so both interfaces operate on the same inventory data.

Figure 3.1 shows the browser, API server, algorithm engine and CSV storage as connected components, with the terminal application attached to the same logic layer.

3.2 Data Flow

A user action in the dashboard sends an API request carrying the user's login token. The Java server checks the token, reloads `inventory.csv` if its modification time has changed, applies the required algorithm, saves any change by writing a temporary file and renaming it over the original, and returns the result as JSON. The dashboard checks for changes every three seconds, so edits made in the terminal appear on the website automatically.

3.3 Module Mapping

- Module 1 (TextHack as a system) → Inventory Search: tokenising queries, correcting typos using edit distance over the product vocabulary, and “did you mean” suggestions.

- Module 2 (String algorithms) → Inventory Search and String Lab: Aho-Corasick for multi-word search, KMP for phrase matching, and Z-function, Rabin-Karp and suffix array with Kasai LCP for comparison and large-file search.
- Module 3 (Advanced dynamic programming) → Replenishment: 0/1 knapsack to select the best restocks within a budget; edit distance (Wagner-Fischer with transpositions) for fuzzy matching.
- Module 4 (Network flow) → Warehouse Balancing: Edmonds-Karp maximum flow over capacity-limited warehouse routes.
- Module 5 (NP-completeness and approximation) → Risk & Priority: risk ranking and greedy set cover (logarithmic approximation) for choosing suppliers.
- Module 6 (Randomised and parallel algorithms) → Cycle Counting: reservoir sampling.
- Module 6 → Large Inventory Processing: parallel map-reduce aggregation, randomised quickselect, and streaming search over 10,000,000 records.

3.4 Algorithm-to-Feature Mapping

The system is organised around user-facing warehouse functions; the algorithms are the mechanisms behind them: M1 and M2 power Inventory Search; M3 powers Replenishment; M4 powers Warehouse Balancing; M5 powers Risk & Priority; and M6 supports Cycle Counting and Large Inventory Processing.

4. IMPLEMENTATION

4.1 Module-Wise Implementation

Inventory Search (Modules 1 and 2)

The query is split into words. Aho-Corasick scans the combined product text once for all words. Words that match nothing are replaced by the closest word in the product vocabulary, measured by edit distance with transpositions (at most one change for words up to three letters, two otherwise), and the search is repeated; the website then offers a “Did you mean” suggestion. Products matching every word are ranked by matched words, with extra weight when KMP finds the whole

phrase in the name and when the query equals the SKU. If no product matches every word, partial matches are shown.

Replenishment (Module 3)

Every product below its minimum stock level is a candidate. The order quantity brings stock to twice the minimum, the cost is quantity times price, and the benefit is quantity times (daily demand + 1), tripled for out-of-stock items. A 0/1 knapsack dynamic programme selects the set with the highest total benefit whose cost does not exceed the budget. The result is compared with a greedy value-per-rupee plan, and the plan can be applied to add the ordered quantities to inventory.

Warehouse Balancing (Module 4)

For each SKU a flow network is built: a source feeds warehouses with surplus stock (stock above twice the minimum), warehouse-to-warehouse edges carry route capacities read from routes.csv, and warehouses below their minimum feed a sink. Edmonds-Karp (breadth-first augmenting paths) computes the maximum flow, which is turned into transfer lines; any shortage that cannot be met is reported. Applying the plan changes stock levels while keeping the total number of units unchanged.

Risk & Priority (Module 5)

Each record receives a risk score from 0 to 100 based on days of stock cover (stock divided by daily demand), with a penalty when below minimum and a score of 100 when out of stock. Products and warehouses are ranked by this score using a randomised quicksort. Choosing the fewest suppliers to contact so that every at-risk product is covered is a set-cover problem, which is NP-hard; a greedy algorithm is used, which guarantees a result within a logarithmic factor of the optimum, and the guarantee for the current data is displayed.

Cycle Counting (Module 6)

Reservoir sampling (Algorithm R) reads the records once and keeps a uniform random sample of the requested size without knowing the total in advance. The user may fix a seed to reproduce a sample, enter the counted quantity for each

item, and log the variance or adjust the stock to the counted value; every count is appended to results/cycle_counts.csv.

Large Inventory Processing (Module 6)

The 10,000,000-record file is generated by a streaming generator in about eight seconds. Aggregation splits the file into byte ranges, one thread per range, each producing warehouse and category totals that are merged at the end. Percentiles of stock quantity are computed with randomised quickselect, products can be searched with KMP, Z-function or Rabin-Karp while streaming, and a reservoir sample can be drawn from the whole file.

4.2 Front-End Logic and UI Rendering

The Warehouse Control Center is a single HTML page with CSS and JavaScript, served by the Java program. After login it provides tabs for Dashboard, Inventory, Replenishment, Balancing, Risk & Priority, Cycle Count, Large Data, String Lab and Modules. The Dashboard shows total products, units, stock value, low-stock, out-of-stock and surplus counts, per-warehouse and per-category tables, and the products needing attention first. The Inventory tab has a search box, filters, sortable columns, pagination, and an add/edit form with validation messages. Result pages show summary figures, notes and tables returned by the server.

4.3 API Integration

The browser calls endpoints under /api/, sending the login token in a request header. Examples are login, dashboard, products (search, sort, filter and paging), products/add, products/update, products/delete, replenish, balance, risk, cycle, cycle/record, strings, and large/generate, large/aggregate, large/search, large/sample and large/percentiles. Replenishment and balancing have an /apply variant that changes the stored inventory. The server runs on port 8080 by default; errors are returned as JSON messages with suitable status codes (for example 401 for no login, 403 for a staff user deleting, and 409 for a duplicate SKU in a warehouse).

4.4 CSV Data Processing

inventory.csv is the common source of inventory information for both the website and the terminal. On every operation the program compares the file's modification time and size with the last copy it loaded and reloads when they differ. Every save writes inventory.csv.tmp and then renames it over the original, so the file is never left half-written. The large file is never loaded whole: it is read as a stream, or in byte ranges by parallel threads that each start at the first full line inside their range.

4.5 Pseudocode

1. LOAD inventory.csv (create sample data on first run); PARSE each line; skip invalid lines
2. LOGIN: check username and salted SHA-256 password hash; issue a token
3. ON add/update/delete: validate fields; reject duplicates; change the record list; SAVE atomically
4. IF Inventory Search: tokenise; Aho-Corasick scan; correct unmatched words by edit distance; rank; return
5. IF Replenishment: build candidates below minimum; compute cost and benefit; run 0/1 knapsack; return plan; optionally APPLY
6. IF Warehouse Balancing: for each SKU build source–warehouse–sink network; run Edmonds-Karp; return transfers; optionally APPLY
7. IF Risk & Priority: compute risk scores; rank with randomised quicksort; run greedy set cover over suppliers
8. IF Cycle Counting: reservoir-sample k records; record counted quantity and variance
9. IF Large Inventory Processing: split the file into byte ranges; threads aggregate; merge; or stream for search, quickselect and sampling
10. RETURN the result to the terminal (as a table) or to the browser (as JSON)

5. RESULTS

5.1 Test Environment

All measurements were taken with Java 21 on a Linux virtual machine with one CPU core, using the 2,804-record `inventory.csv` and a generated 10,000,000-record `inventory_large.csv` (951,430,312 bytes). The website was exercised through its HTTP API by a test script, and the algorithms by the built-in self-tests. Times on the team's own computers will differ, and with several CPU cores the parallel aggregation will be faster than reported here.

5.2 Functional Results

The self-test command (`java -cp out Main test`) ran 23 tests and all passed. The tests compare KMP, Z, Rabin-Karp, Aho-Corasick and the suffix array with naive search on random strings; check the Kasai LCP array; verify edit distance on known cases; compare the knapsack with a brute-force optimum on 150 random instances; confirm a maximum flow of 23 on the standard textbook network and that flow value equals minimum-cut capacity on 100 random networks; check set cover, randomised quicksort and quickselect, and uniformity of reservoir sampling; and run the large-data pipeline on 200,000 records (4-thread aggregation equals 1-thread aggregation, and all three string algorithms agree). A separate API script made 43 checks of login, permissions, add/update/delete, validation, every module, and the large-data endpoints; 42 passed and the one failure was a wrong expectation in the test itself (a word that is only a prefix of a product word matches without correction). Applying a balancing plan kept the total number of units unchanged, and applying a replenishment plan added exactly the ordered units. A product added in the terminal appeared on the website and was then deleted from the website.

- Inventory Search: misspellings such as “haevy duty”, “gloevs”, “claw hamer” and “safty helmet” are corrected and the right products are returned
- Replenishment: with a budget of 50,000 the plan chose 7 items costing 49,919
- Warehouse Balancing: on the sample data the plan contained 806 transfer lines moving 30,832 units

- Risk & Priority: products ranked by score, with a greedy supplier plan that reaches 100% coverage of at-risk products
- Cycle Counting: a sample of the requested size without duplicates, reproducible with the same seed
- Large Inventory Processing: aggregation, search, percentiles and sampling over 10,000,000 records

5.3 Performance Observations

Measured on the 10,000,000-record dataset (single-core test machine): generating the dataset took 8 s; parallel aggregation with 4 threads 5.7 s (about 1.75 million records per second); streaming search with KMP 3.1 s (“hex bolt” matched 276,863 records) and with Rabin-Karp 6.3 s; percentiles by quickselect 2.6 s; a reservoir sample 1.2 s. All 23 self-tests, including a 200,000-record pipeline, ran in about 3 s. The four-thread aggregation ran on one core, so it shows the correctness of the parallel design rather than a speed-up.

5.4 Screenshots and Analysis

Insert screenshots of the Dashboard, Inventory (including a typo search with the “Did you mean” banner), Replenishment, Balancing, Risk & Priority, Cycle Count, Large Data, and String Lab pages taken from the running application. Each figure should have a chapter and figure number and a descriptive caption as listed in the List of Figures.

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

Several problems were found and corrected during development. First, the project was originally planned with a React and Node.js front end and a database, then with several separate Java files; it was simplified to one Java source file, a plain HTML page and CSV storage so that it runs in VS Code with only a JDK. Second, testing showed that a misspelling such as “copr wire” was not corrected because short words were allowed too few edits; the limit was changed so that words of up to three letters allow one edit and longer words allow two. Third, one API test

expected “stainles” to be corrected, but it is a valid prefix of “stainless” and matches without correction, so the test expectation was fixed rather than the program.

6.2 Solutions Applied

Each issue above was retested after the change: the typo test now corrects “copr wire” to “copper wire”, the self-tests (23) pass, and the API checks pass. To keep the website and terminal consistent, the program reloads the CSV when its modification time changes and saves by renaming a temporary file, and the dashboard checks for changes every three seconds.

6.3 Limitations of the System

The project uses CSV storage rather than a production database and is intended for academic and local execution. All data is synthetic. Login uses salted hashed passwords and in-memory session tokens, which is suitable for a demonstration but not for production. The locking that protects the CSV file works inside one running program; if the terminal and the website write at exactly the same moment from separate processes, one change could overwrite the other. Balancing uses fixed route capacities from a file, risk uses a simple days-of-cover formula, and replenishment considers the 400 best benefit-per-rupee candidates. The 10,000,000-record file is about 0.95 GB and must be generated on each computer. Timings were measured on a single-core virtual machine, and the browser pages have not been tested on every browser.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Warehouse Inventory Management and Optimization System shows how algorithmic techniques from the course can be integrated into a practical warehouse-control application. By connecting a CSV inventory, a hand-built Java algorithm engine, an API layer, a terminal application, and a browser dashboard, the project provides one environment for search, replenishment, stock balancing, risk prioritisation, cycle counting, and processing of a 10,000,000-record dataset.

All six course modules are executable, the website and terminal share the same storage, and 23 automated tests pass.

7.2 Future Work

- Replace or complement CSV storage with a production database.
- Strengthen authentication (password change, stronger session handling) and add finer permissions.
- Add richer warehouse analytics and historical trends.
- Introduce stronger real-time synchronization and monitoring.
- Expand optimization constraints and warehouse-capacity models.
- Add automated performance benchmarking and regression testing.
- Deploy the system as a production web service.

7.3 Skills Learned During Development

The project provides experience in algorithm implementation (string matching, dynamic programming, network flow, approximation, randomised and parallel algorithms), Java programming without library collections in the engine, CSV and large-file processing, building a small HTTP API, front-end development, writing automated tests, debugging, and translating academic algorithms into practical software features.

REFERENCES

The following sources were used for the algorithms and the Java implementation.

- [1] Oracle, Java SE Documentation, including the `com.sun.net.httpserver` package, used for the implementation.
- [2] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast pattern matching in strings,” SIAM J. Computing, 1977; A. V. Aho and M. J. Corasick, CACM, 1975; R. A. Wagner and M. J. Fischer, J. ACM, 1974 (string matching and edit distance).

- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022; J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005; R. K. Ahuja, T. L. Magnanti and J. B. Orlin, Network Flows, Pearson, 1993 (dynamic programming, network flow, approximation).
- [4] J. S. Vitter, “Random sampling with a reservoir,” ACM Trans. Mathematical Software, 1985; Cormen et al., Introduction to Algorithms (randomised quickselect and parallel algorithms).